# Depth Charge — Viva Guide (Ali)

*Common overview plus individual module questions*

Course: EL-2003 COAL Lab, Fall 2026

Team: Muhammad Anas (25K-0899), Syed Alyan Hussain (25K-0648), Ali Raza (25K-0827)

Read this page first. Then read your own member guide and skim the other two. The viva can ask about the whole program, not only your folder.

## 1. Explain the project in 30 seconds

> *Depth Charge is a single-player 80 × 25 console game in 32-bit x86 assembly. I move a submarine, fire torpedoes, and avoid falling depth charges. Some charges are hidden mines. Sonar reveals them for two seconds. The game has score, three lives, nine levels, pause, sound, and a saved high score. We used MASM, Irvine32, fixed arrays, procedures, and a simple game loop.*

Do not say it uses DOS video memory or BIOS interrupts. It is a Windows console program. Irvine32 and Windows API calls provide console input/output, timing, files, and sound.

## 2. The five jobs in every frame

Read keys -> Move objects -> Check hits -> Draw changed cells -> Wait

1. **Input:** `HandleInput` checks waiting keys without blocking.
2. **Update:** torpedoes move up; charges sometimes move down; a new charge may spawn; explosions age.
3. **Collisions:** compare object coordinates in memory, not screen pixels.
4. **Render:** build a complete frame in `ScreenBuf`, then compare it with `PrevBuf`.
5. **Delay:** sleep for the remaining part of the 80 ms frame target.

GameLoop in src/game.asm controls this order. gameState decides whether object updates run.

| State | Value | Meaning |
|---|---|---|
| **`ST_MENU`** | 0 | Menu is outside the gameplay loop |
| **`ST_PLAYING`** | 1 | Normal updates and drawing |
| **`ST_SONAR`** | 2 | Objects frozen; hidden mines visible |
| **`ST_PAUSED`** | 3 | Objects frozen; pause message visible |
| **`ST_GAMEOVER`** | 4 | Save record and show final score |
| **`ST_EXIT`** | 5 | Leave the program |

P remembers whether the game was playing or in sonar. If sonar was active, its start time is shifted forward by the paused duration, so pausing does not consume the two-second reveal. The ten-second recharge is wall-clock time and continues during an ordinary pause.

### 3. The one structure everyone uses

```
GameObject STRUCT
   xPos    BYTE 0
   yPos    BYTE 0
   active  BYTE 0
   hidden  BYTE 0
GameObject ENDS
```

- Size = **4 bytes**. `TYPE GameObject` is 4.
- `active = 0` means the slot is free.
- `hidden = 1` means a mine is invisible during normal drawing.
- Hidden does **not** mean inactive. Collision code still checks it.
- Torpedoes: 8 × 4 = 32 bytes.
- Charges: 15 × 4 = 60 bytes.
- Explosions: 5 × 4 = 20 bytes, plus five separate timer bytes.

Example: if ESI points to Charges[0], add esi, TYPE GameObject points to Charges[1]. Adding 1 would point inside the first object, not to the next object.

### 4. Screen memory is not the Windows video card

Each screen cell is one WORD:

```
low byte  = ASCII character
high byte = colour attribute
cell index = row * 80 + column
byte offset = cell index * 2
```

Example: row 5, column 10 → (5 × 80 + 10) × 2 = 820 bytes from the start of ScreenBuf.

The buffer has 80 × 25 = 2000 WORDs = 4000 bytes. This resembles old text-mode layout, but it is ordinary program memory. Gotoxy, SetTextColor, and WriteChar update the Windows console. We never write directly to B800h.

The bottom-right cell is not printed because writing there can scroll some consoles. The renderer still copies the blank cell into PrevBuf; it simply never sends that cell to the console.

### 5. Course-outline mapping — what we actually used

| Course topic | Where to show it | Simple explanation |
|---|---|---|
| **IDE, assemble, debug, run** | `Project.sln`, Win32 build | MASM makes an OBJ; linker makes an EXE |
| **Data definition and strings** | `data.inc`, `screens.inc` | BYTE/WORD/DWORD and zero-terminated text |
| **MOV, MOVZX, LEA** | `render.inc`, `hud.inc` | Copy data, extend a byte safely, take an address |

| | | |
|---|---|---|
| **Flags and conditional jumps** | input, collisions, timers | `CMP` sets flags; `JE/JB/JAE/JNZ` choose a path |
| **Indirect/index addressing** | object loops, `ScreenBuf[ebx*2]` | A register holds an address or index |
| **Arrays and loops** | torpedoes, charges, explosions | Fixed slots avoid dynamic memory allocation |
| **Procedures and stack** | every module; `USES`, `CALL`, `RET` | Reusable code; save/restore registers |
| **Stack frame and LOCAL** | `PutNumber` | Temporary digit string belongs to this call |
| **Shift and rotate topic** | `SHL` for colour, `SHR` in random | Shift bits left/right; rotate is not needed |
| **Multiply/divide** | row offset, LCG, digits, level | `MUL/IMUL` and `DIV` with correct EDX setup |
| **String instructions** | `REP STOSB/STOSW`, `REP MOVSD` | Clear arrays/buffers and copy a frame |
| **Structures and scaling** | `GameObject`, explosion indexing | `TYPE` gives the element size |
| **Macros** | `mPlaySound` | Expand a small save/Beep/restore pattern |
| **OS services** | Irvine32 and Windows API | Console, timing, files, and Beep |

Not every listed course topic must appear in one project. This game does not use recursion, inline assembly, extended multiword arithmetic, or the optional REPE CMPSW renderer. Saying that honestly is better than inventing a feature.

### 6. Questions every member should answer

Why x86 and not x64?

The course library is Irvine32. Its procedures and library are 32-bit, so the project uses Win32 and 32-bit registers.

Why is there one .asm file but many .inc files?

INCLUDE inserts source text before assembly. It split work by owner without creating separate linking contracts. There is still one assembled source unit.

Why not call Clrscr every frame?

Clearing and redrawing everything causes flicker and unnecessary console calls. We clear only the memory buffer and print changed cells.

Why can a hidden mine hit me?

The renderer checks hidden; the collision procedure checks active and coordinates. The object remains in the array.

Why does DIV need EDX = 0 here?

Unsigned 32-bit DIV divides the 64-bit value EDX:EAX. For these small unsigned dividends, the upper half must be zero. Otherwise the result can be wrong or the quotient can overflow.

What does USES eax ecx do?

MASM generates pushes at procedure entry and matching pops before return. The caller's values in those registers are preserved. It does not automatically preserve registers not listed.

Does USES preserve flags?

No. Do not assume ZF or CF survives a procedure call unless its contract explicitly says so.

Why is the frame rate a target, not a guarantee?

The OS schedules other programs, console calls take time, and Beep blocks. The loop waits for the remaining time when possible; it cannot guarantee hard real-time deadlines.

Where is the high score saved?

In highscore.dat in the current working directory. It stores one four-byte DWORD in little-endian binary form, not readable decimal text.

What happens when I press ESC during a round?

The round ends and the menu returns. The current score is not saved. Saving happens at game over, as described in the instructions.

### 7. Three-minute demonstration script

6. Show the menu and high score. Open instructions, return, then start.
7. Move with both letters and arrow keys. Touch both boundaries.
8. Fire several torpedoes. Point out the small firing gap.
9. Let charges fall. Explain that `*` is visible and hidden mines are in memory.
10. Press `SPACE`. Point to `SONAR ACTIVE` and any green `!` marks.
11. Press `P` during sonar, wait, then resume. Explain the remembered state.
12. Hit a charge if possible; show the explosion and score change.
13. Lose three lives or use the debugger to demonstrate `LoseLife` honestly.
14. Show game over and the saved high score. Restart the program to show persistence.
15. Start a new round and show score 0, lives 3, level 1, and empty arrays.

If random play does not produce a hit during the short demo, use a prepared debugger state and say it is a controlled test. Do not pretend a random event was guaranteed.

### 8. Debugger questions you may be asked

- Put a breakpoint at `DoPlaying` and step through one frame.
- Watch `subX`, `score`, `lives`, `level`, and `gameState`.
- Open Memory at `Charges` and identify four bytes per object.
- Open Memory at `ScreenBuf` and find a known character/colour WORD.
- Break at `Hit` in `collision.inc` and explain which two objects matched.

- Break at `Rand` and show `EDX` cleared before `DIV`.
- Break at `SaveHighScore` and show the four-byte write.

### 9. Honest preparation checklist

- [ ] I built the exact commit on my own/lab Windows PC.
- [ ] I can draw the frame loop without looking.
- [ ] I can explain every procedure in my own folder.
- [ ] I read the other two members' summaries.
- [ ] I ran the manual tests and recorded real results.
- [ ] I personalized my report and disclosed AI assistance.
- [ ] I can explain a limitation instead of guessing.

## Viva guide — Ali Raza (25K-0827)

Your area: random numbers, charges/mines, menus, score/lives/levels, sound, high-score file.

Files: src/ali/.

Read 00-common.md first. Be ready to explain unsigned division and the difference between a hidden object and an inactive object.

### 1. Your 30-second introduction

> *I provide the game rules and content. My random generator chooses charge columns and hidden mines. Timers control spawning and falling. I update score, lives, and level, show the menu and game-over screen, play short Beep effects, and save the best score to a four-byte file.*

### 2. Procedure map

| Procedure | Input | Result |
|---|---|---|
| **`SeedRandom`** | Clock | Initializes `seed` |
| **`Rand`** | `EAX` = positive range | `EAX` = 0 through range−1 |
| **`SpawnCharge`** | Spawn timer, free charge slot | New visible charge or hidden mine |
| **`UpdateCharges`** | Fall timer | Moves active charges down every N frames |
| **`AddScore`** | `EAX` = points | Adds score and recalculates level |
| **`LoseLife`** | Submarine hit | Decrements life; game over at zero; otherwise recenter |
| **`ApplyLevel`** | `level` | Copies fall/spawn delays from tables |
| **`ShowMenu`** | Key queue | Start, instructions, or exit state |
| **`ShowInstructions`** | None | Rules/controls; waits for a key |
| **`GameOverScreen`** | Final score | Saves record, shows result, waits, returns to menu |
| **`Play...Sound`** | Event | Blocking Windows Beep tone(s) |
| **`LoadHighScore`** | File | Valid DWORD or zero |

| | | |
|---|---|---|
| `**SaveHighScore**` | Score vs record | Writes only a new record; reports failure |

### 3. Random numbers

The generator is a small linear congruential generator (LCG):

```
seed = (seed * 1103515245 + 12345) mod 2^32
value = seed >> 16
result = value mod range
```

- `MUL ecx` produces `EDX:EAX`; keeping low `EAX` gives modulo `2^32` naturally.
- `SHR eax,16` uses upper bits, which are less visibly patterned in this simple LCG.
- `XOR edx,edx` clears the upper dividend before `DIV ebx`.
- The remainder in `EDX` becomes the return value in `EAX`.
- Range 78 gives 0–77; adding `MIN_COL` gives columns 1–78.
- Range 5 gives 0–4; result 0 marks about 20% as hidden.
- A zero range returns zero instead of dividing by zero.

This is pseudo-random game logic, not cryptography. Modulo can introduce a tiny bias; that is acceptable for this course game and not worth a complex rejection algorithm.

### 4. Charge timers

spawnTimer counts frames until the next spawn. At level 1, spawnDelay=14, so a new charge appears about every 1.12 seconds.

fallTimer counts frames until all active charges move one row. At level 1, fallDelay=6, so a charge steps about every 480 ms.

A charge starts at row 2. At row 24 it becomes inactive. Submarine collision is checked at row 23 before it reaches the sea bed.

If all 15 slots are full, spawning is skipped. The game does not allocate more memory.

### 5. Score, lives, and levels

```
level = score / 200 + 1, maximum 9
```

Examples:

| Score | Level |
|---|---|
| **0–199** | 1 |
| **200–399** | 2 |
| **400–599** | 3 |
| **1600 or more** | 9 |

ApplyLevel uses index level−1:

```
fall delays:  6, 5, 5, 4, 4, 3, 3, 2, 2
spawn delays: 14,12,11,10, 9, 8, 7, 6, 5
```

Smaller number = fewer frames to wait = faster game. The current countdown is not forcibly reset on level-up; the new delay applies when that timer next reloads.

LoseLife plays the life-lost tones, decrements lives, and either sets game over or recenters the submarine at column 40. Anas's collision code limits submarine hits to one per frame.

## 6. Sound macro

```
mPlaySound MACRO frequency, duration
   pushad
   INVOKE Beep, frequency, duration
   popad
ENDM
```

A macro is expanded by the assembler; it is not a runtime procedure call. PUSHAD/POPAD save the eight general-purpose registers around the Windows call.

Beep blocks until the tone finishes:

| Event | Tone pattern |
|---|---|
| **Fire** | 1200 Hz, 20 ms |
| **Explosion** | 150 Hz, 30 ms |
| **Life lost** | 300 Hz 120 ms, then 200 Hz 180 ms |
| **Game over** | Four descending notes |
| **Sonar** | 400, 500, 600, 700, 800 Hz, 50 ms each |

Normal-play tones are short. Sonar already freezes objects. Life-lost/game-over tones can lengthen a frame; the game does not claim hard real-time timing. Muting audio does not change the rules.

## 7. High-score file trace

The file contains exactly one DWORD, four bytes, little-endian. Score 250 is stored as bytes FA 00 00 00, not the text "250".

### Load

16. Set `highScore=0` first.
17. `OpenInputFile`; missing file is a normal first-run case.
18. Read four bytes.
19. Check carry flag and byte count.
20. Close the handle.
21. A short/unreadable file leaves zero.

### Save

22. Clear `newRecord` and `saveFailed`.
23. Compare score with the old record using unsigned `JBE`.
24. If not higher, return.
25. Create/truncate the file and write four bytes.
26. Check the written count and close the handle.

27. Only after a successful write update `highScore` and set `newRecord`.
28. On failure, keep the old in-memory record and show a message.

This fixes the guide's equality-message bug: tying an old record is not announced as a new record.

The simple version does not use an atomic temporary-file rename. A power loss or disk-full error during overwrite can damage the small file; the next load detects a short file and returns zero. That is a stated limitation, not a hidden feature.

### 8. Menus and game over

Menus use direct console output because objects are not moving. Delay 30 in the wait loop avoids a full-CPU busy loop. ReadKeyFlush discards old gameplay/menu keys at screen transitions.

Game over saves first, then displays final score, high score, and only the correct new-record/error message. It waits for a fresh key and returns to the menu. ESC during play intentionally does not save.

### 9. Likely questions and short answers

Why must EDX be zero before DIV? The dividend is EDX:EAX; these unsigned values fit in EAX, so the upper half is zero.

What if the file does not exist? High score starts at zero; no crash.

Why binary instead of text? One DWORD is simple and fixed-size. Text would require parsing and formatting.

Why save only at game over? That is the proposed rule and keeps file writes out of the frame loop.

Why is hidden separate from active? A mine must remain logically present while visually hidden.

Why lookup tables for levels? Nine fixed difficulty values are easier to read and adjust than nine branches.

Why does PUSHAD not save flags? It saves general registers only. This sound macro does not promise to preserve flags.

What does your module use from Anas? Shared state, subX, game-loop calls, and collision events that call AddScore/LoseLife.

What uses your data in Alyan's module? The renderer draws Charges; the HUD reads score/lives/level and sonar readiness.

### 10. Practical task before viva

- Call `Rand(78)` repeatedly in the debugger and check the range.
- Watch a charge's four bytes change from inactive to active.
- Set score to 190, add 10, and show level 2 and the new table delays.
- Show the four bytes of score 250 in Memory and in `highscore.dat`.
- Delete the file, restart, and show high score zero.

- Explain why a tied score does not print `NEW HIGH SCORE`.

### 11. Limitations you can state confidently

- The random generator is for gameplay, not security.
- Sound is blocking and device-dependent.
- The high-score write is simple, not atomic.
- The HUD's five-digit display is separate from the DWORD score storage.